

Jalapeño by OpenAI



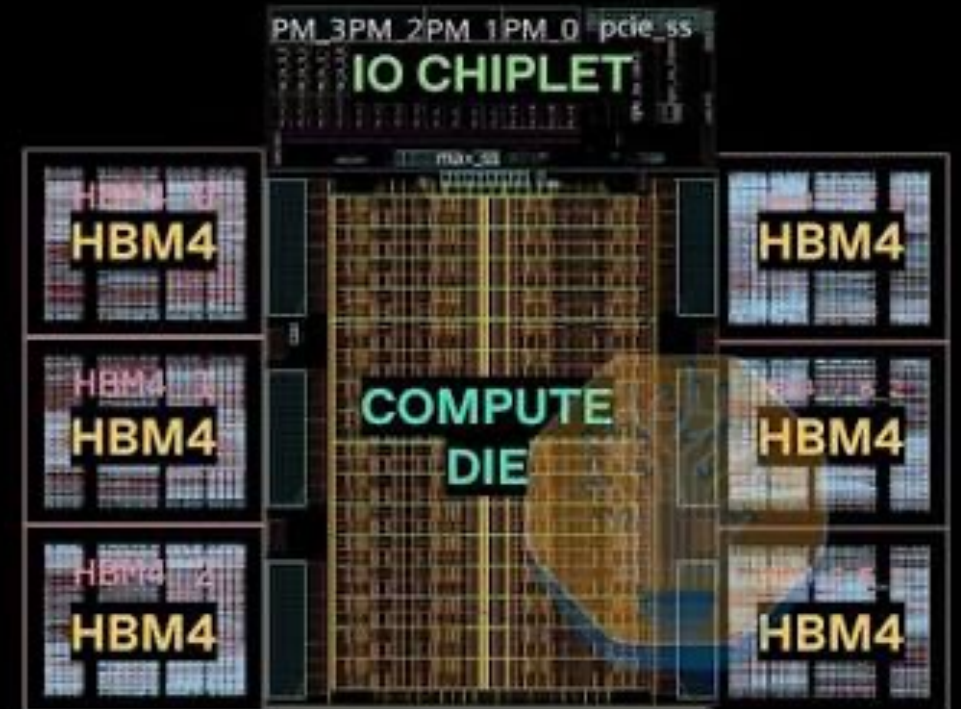
AI Designed the Chip. AI Programs the Chip. The Chip Runs AI.

OpenAI used its own models to design circuits and write code. Concept to tapeout in nine months.

For selected model blocks — AI-written code ran 1.5 to 1.8x faster than human-expert code.

Deploying across OpenAI by end of 2026. Generation 2 already in development.

The AI hardware race just got a new competitor. And it built itself.



Richard Ho brought deep custom-accelerator experience to the program

Before OpenAI

Richard Ho helped lead Google's TPU work before joining OpenAI as VP of Hardware in November 2023. That background mattered because Jalapeño combined a new architecture, software stack, and accelerated schedule.

“They can explore a lot more paths.”



[Home](#) > [Publications](#) >

In-Datcenter Performance Analysis of a Tensor Processing Unit

Norman P. Jouppi · [Cliff Young](#) · Nishant Patil · [David Patterson](#) · Gaurav Agrawal · Raminder Bajwa · Sarah Bates · Suresh Bhatia · Nan Boden · Al Borchers · Rick Boyle · Pierre-luc Cantin · Clifford Chao · Chris Clark · Jeremy Coriell · Mike Daley · Matt Dau · Jeffrey Dean · Ben Gelb · Tara Vazir Ghaemmaghami · Rajendra Gottipati · William Gulland · Robert Hagmann · C. Richard Ho · Doug Hogberg · John Hu · Robert Hundt · Dan Hurt · Julian Ibarz · Aaron Jaffey · Alek Jaworski · Alexander Kaplan · Harshit Khaitan · Andy Koch · Naveen Kumar · Steve Lacy · James Laudon · James Law · Diemthu Le · Chris Leary · Zhuyuan Liu · Kyle Lucke · Alan Lundin · Gordon MacKean · Adriana Maggiore · Maire Mahony · Kieran Miller · Rahul Nagarajan · Ravi Narayanaswami · Ray Ni · Kathy Nix · Thomas Norrie · Mark Omernick · Narayana Penukonda · Andy Phelps · Jonathan Ross · ISCA (2017) (to appear)

[Download](#)

[Google Scholar](#)

[Copy Bibtex](#)

Jalapeño optimizes tokens per second per watt across three inference phases

The goal is high energy efficiency without missing the request's latency target.

1 Prefill

The accelerator processes all prompt tokens in parallel. Large matrix multiplications and long-context attention make this phase compute intensive.

2 Draft

A smaller model proposes several candidate tokens one step at a time. The batch is extremely small, so fixed launch, synchronization, and communication delays are difficult to hide.

3 Verification

The main model checks several candidates together. This creates more parallel computation, but it still reads weights and the attention state from HBM; mixture-of-experts routing can add communication bursts.

Optimization goal: maximize useful tokens per second per watt while the complete request stays within its latency target.

This three-phase workload assumes speculative decoding; other serving systems may use a different method.

Li Shang

Interactive generation makes a large accelerator difficult to use efficiently

Small batches expose costs that a large batch can spread across many tokens.

Large batch

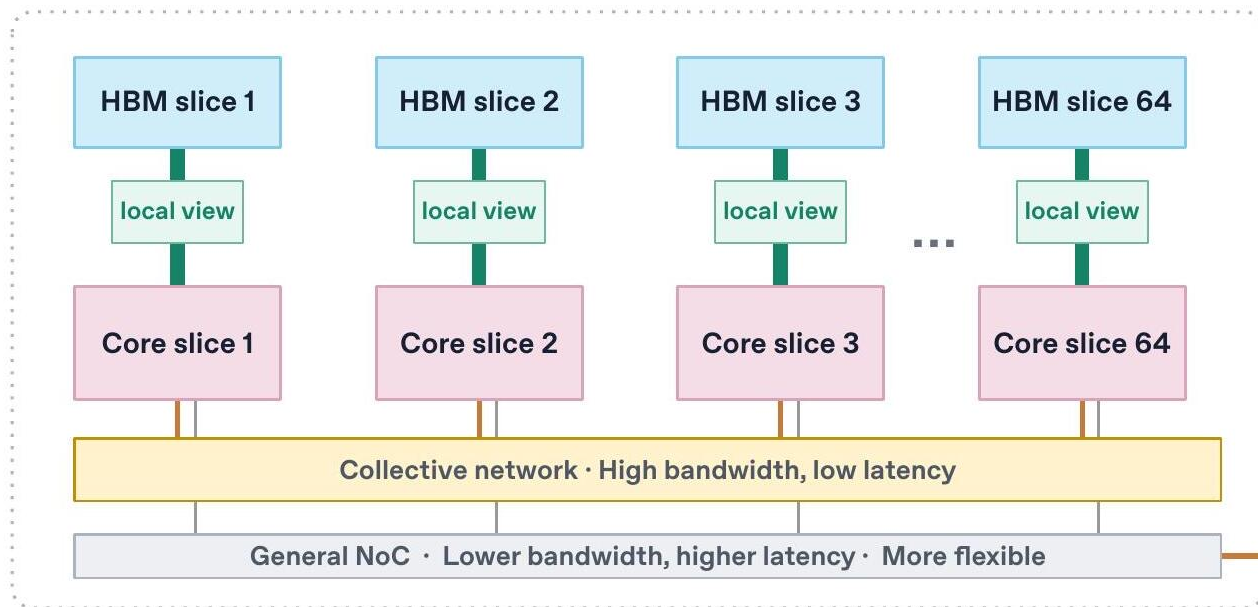
Many requests run together. They reuse loaded weights across more tokens, keep more matrix units busy, and spread kernel-launch and device-coordination costs across more useful work.

Interactive service

The service cannot wait to collect a large batch. It still launches kernels, reads weights, and coordinates devices, but fewer generated tokens share those costs. More hardware therefore sits idle.

Design requirement: maintain high tokens per second per watt across the latency-throughput curve, including the small-batch points that matter to interactive users.

Three architecture features reduce data movement and communication delay



Local HBM path

Problem: a shared memory path adds contention. Each compute slice receives a local view of one HBM slice, so common operands take a shorter path.

Dedicated collective network

Problem: tensor parallelism repeatedly exchanges data. A specialized network gives collective operations predictable bandwidth and latency.

Separate general network

Problem: specialized collectives cannot carry every message. A general network preserves flexibility and connects the chip to the scale-up fabric.

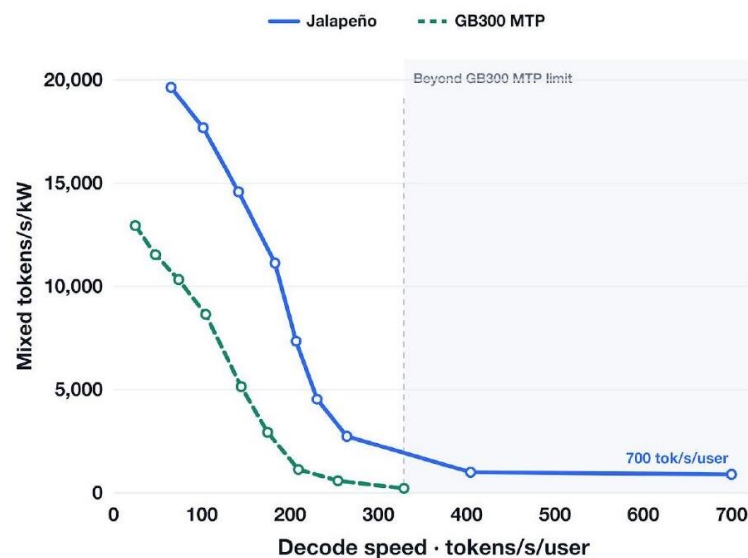
Published specifications: 216 GiB HBM4 • 15.4 TB/s HBM bandwidth • 13.4 PFLOP/s MXFP4 • 700 W

Reported results improve both efficiency and user-visible latency

InferenceX · DeepSeek R1 MXFP4 · nominal 8k/1k · Jalapeño STP vs. GB300 MTP · TDP: 700 W / 1,400 W

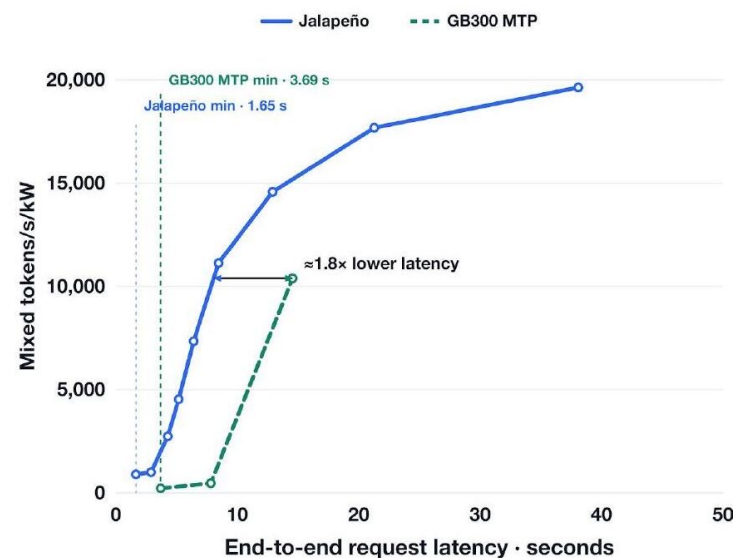
MIXED THROUGHPUT VS. INTERACTIVITY

Efficiency across decoding speeds



MIXED THROUGHPUT VS. END-TO-END LATENCY

Efficiency across full-request latency



How to read the plots

Left: moving upward means more total work per kilowatt; moving right means each user receives tokens faster.

Right: moving upward means more work per kilowatt; moving left means the complete response arrives sooner.

Across three public models: 1.5-1.9× higher peak performance per watt, 1.7-3.6× lower end-to-end latency, and 2.1-4.1× higher interactivity.

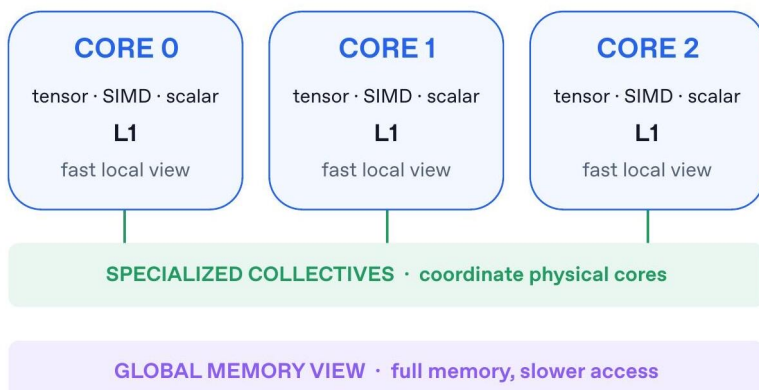
Evidence boundary: OpenAI supplied the results; SemiAnalysis observed selected engineering-sample runs. Complete AgentX and production reliability results were not public.

Li Shang

Gluon exposes the hardware details that high-performance kernels must control

Gluon programs each core like a thread block; specialized collectives coordinate across cores.

GLUON PROGRAMS RUN ON PHYSICAL CORES



CORE = THREAD BLOCK

Multiple engines share L1 and a fast local memory view.

SPECIALIZED COLLECTIVES

Coordinate work and data across physical cores.

TENSORINFO

Captures tensor layouts and physical placement explicitly.

Gluon

OpenAI's kernel language, built on Triton, exposes matrix instructions, data movement, synchronization, and prefetching.

Linear Layouts

A layout algebra maps tensor elements to registers, memory banks, and physical cores, and transforms those mappings.

TensorInfo

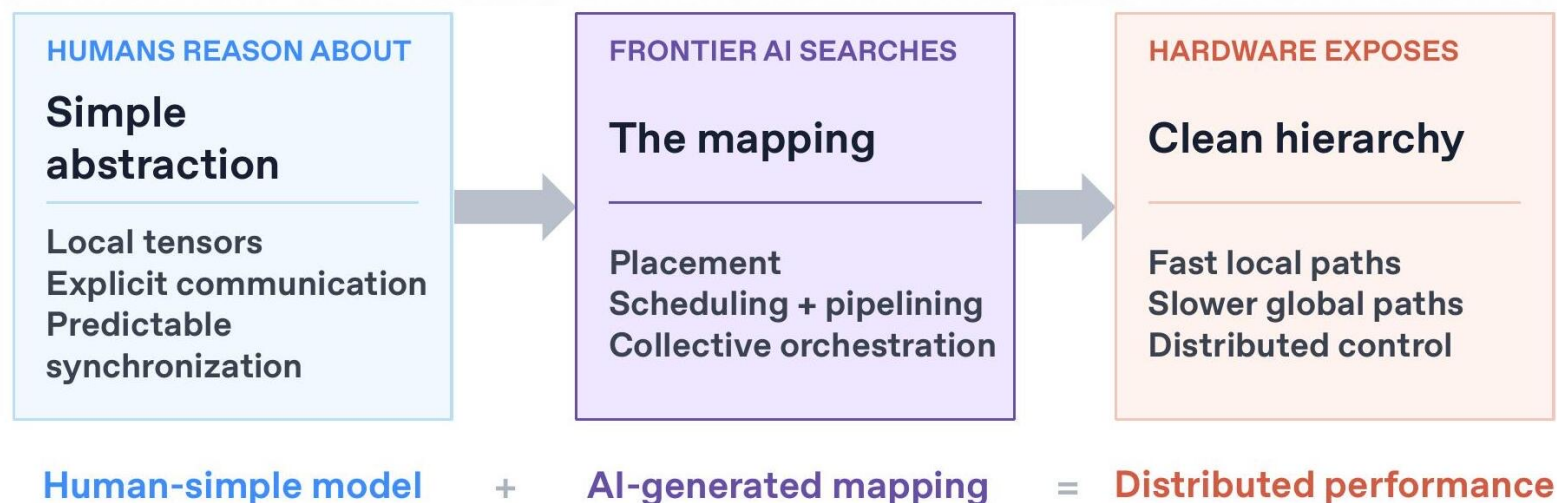
The program carries the chosen tensor layout and physical placement explicitly instead of hiding them in the compiler.

Spatial model

Work is mapped onto physical cores that behave like persistent thread blocks with tensor, vector, and scalar engines.

The result is assembly-like code with explicit tiling, scheduling, prefetching, synchronization, and data movement. This control enables optimization—but creates a search space that is difficult to explore manually.

AI searches the mapping from a correct program to the distributed chip



Problem

A correct kernel does not specify the fastest placement and schedule for every model shape.

Search variables

AI chooses placement, tiling, overlap, prefetch timing, synchronization, and collective communication patterns.

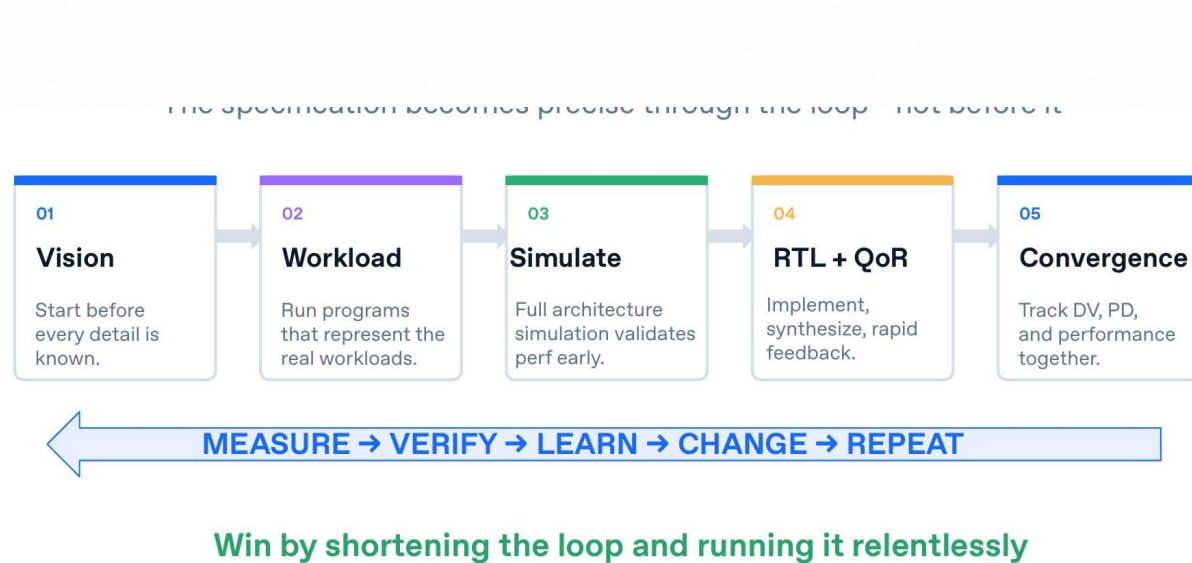
Trusted evaluators

Tests protect correctness; the compiler, simulator, synthesis, formal checks, physical-design tools, and silicon measure quality.

AI is the meta-optimizer across specialist tools; it does not replace those tools.

Li Shang

Hardware and software converge through a short measurement loop



1. Vision chooses the workload and system objective.

2. Representative programs reproduce the real workload.

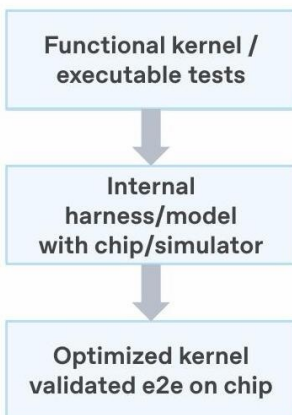
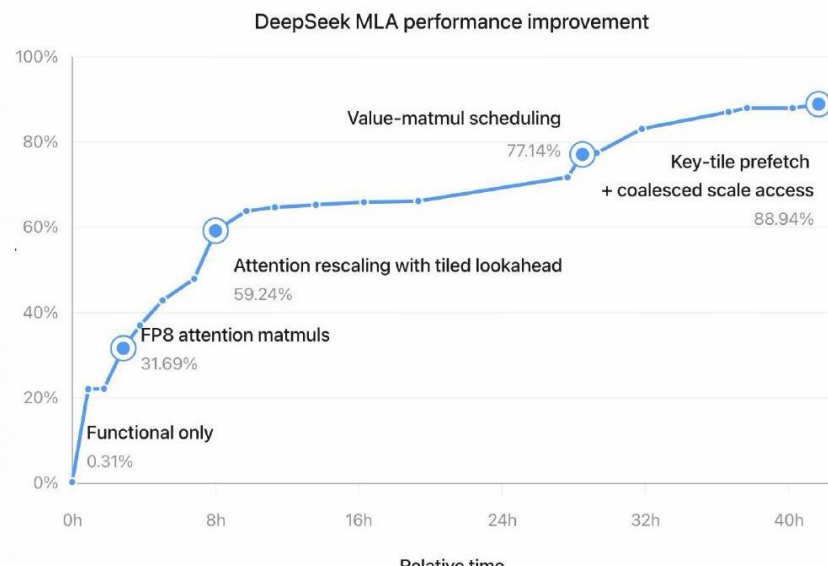
3. Architecture simulation predicts cycle-level performance.

4. RTL and quality-of-result checks measure timing, power, and area.

5. Verification, physical design, software, and hardware measurements drive convergence.

AI proposes more alternatives and helps inspect failures; measurement and verification decide which alternatives survive.

Verified search produced large improvements in selected kernels



DeepSeek kernels

AI drove most of the optimization from a functional baseline.

GPT-OSS blocks

1.5-1.8× faster than the existing expert-written implementation on attention and MoE blocks.

How the loop works

The process starts with a correct kernel and executable tests. AI changes tiling, scheduling, prefetching, and data access. Tests and a custom sanitizer reject invalid candidates; measured performance becomes feedback for the next proposal.

Measured example

DeepSeek attention rose from 0.31% to 88.94% of the estimated hardware limit in about 40 hours. Selected GPT-OSS blocks were 1.5-1.8× faster than prior expert-written implementations.

Claim boundary: these results describe selected kernels and blocks, not the complete model.

Li Shang